

University of Warsaw
Faculty of Economic Sciences

Adam Foster
Student ID: 457898

DSGE Models in Systematic Trading Strategies

Master's Thesis
programme: Quantitative Finance

Thesis completed under the guidance of
Marcin Chlebus (Ph.D.)
Department of Data Science
WNE UW

Warsaw, May 2026

Supervisor declaration

I declare that this Master's thesis has been prepared under my guidance and that it meets the criteria for submission with the goal of obtaining a Master's degree certificate.

I declare that my involvement in this research paper amounts to%, whilst the abstract has been completed entirely by the student.

Date

Supervisor signature

Student declaration

I declare that this Master's thesis has been written by me and that it does not contain information obtained via disallowed means.

I declare that this thesis has not previously been used to obtain any other degree certificate.

I declare that this version of the thesis is identical to the digital version of the thesis.

I declare that my involvement in this research paper amounts to%, whilst the abstract has been completed entirely by me.

Date

Student signature

Abstract

TO BE UPDATED

Keywords

DSGE, systematic, trading, strategy, momentum, mean-reversion
(DSGE, systematyczna, strategia, trading, momentum, mean-reversion)

Field of work (Erasmus code)

Economics (14300)

Thematic Classification

Tytuł pracy w języku polskim

Modele DSGE w zastosowaniu w systematycznych strategiach tradingowych

CONTENTS

1. INTRODUCTION.....	4
2. LITERATURE REVIEW	6
3. DATA.....	9
3.1. Reproducibility	10
4. METHODOLOGY	11
4.1. RBC Model	11
4.2. Realised GDP.....	14
4.3. Signal Generation	14
4.4. Extensions and Variations	16
4.5. PnL Calculation and Performance Metrics	17
4.6. Buy-and-Hold Benchmark.....	17
5. RESULTS.....	18
6. LIMITATIONS AND IMPROVEMENTS.....	19
7. CONCLUSION	20
BIBLIOGRAPHY	21
APPENDICES.....	23
Appendix A: Framework Setup	23
Appendix B: DSGE Strategy Code.....	23
Appendix C: Basic RBC Model YAML Code	49

1. INTRODUCTION

The search for trading strategies is extensive and spans multiple asset classes. Both discretionary and systematic strategies have been researched, tested and implemented by various practitioners and professionals associated with the field of finance. Within the early days of the systematic category, simple investment decisions otherwise performed by analysts and traders were automated (Chan, 2013; Narang, 2013). This gradually became more sophisticated and soon took advantage of increases in computing power and reduced latency to target market phenomena at high frequency (Aldridge, 2013). Such strategies generally attempted to exploit market microstructure nuances often at tick level (Hasbrouck, 2007; O'Hara, 1995). Systematic strategies have become more complex over time, most recently taking advantage of ever-increasing volumes of data and further leaps in computing ability that can help forecast asset price changes, often departing from economic theory in favour of raw statistical accuracy (Gu, Kelly, & Xiu, 2020; López de Prado, 2018).

In a separate world, modern macroeconomics has largely been shaped by a bottom-up style of modelling economic interactions between agents in the economy, characterised as Dynamic Stochastic General Equilibrium (DSGE) modelling (Kydland & Prescott, 1982; Woodford, 2003; Galí, 2015). In one of its basic forms in the Real Business Cycle (RBC) model, households interact with firms. A large number of households seek to maximise their lifetime utility by balancing consumption and leisure. They supply labour and save by investing in capital, which they then rent to firms. Firms maximise profit. They hire labour and rent capital to produce goods based on the available technology and prevailing competitive wages and interest rates (King, Plosser, & Rebelo, 1988). This relationship has its equilibrium and can be affected by stochastic shocks, for example to total factor productivity (TFP) (Kydland & Prescott, 1982). The modelling framework allows for an elegant depiction of the interaction between entities in the economy and to quantify the impact of macroeconomic shocks. Furthermore, DSGE models can be calibrated to suit the characteristics of particular economies (Smets & Wouters, An Estimated Dynamic Stochastic General Equilibrium Model of the Euro Area, 2003). As is broadly the case in economics, its use case is primarily for inference with the added benefit of forecasting impulse response functions (IRF) of macroeconomic variables under aforementioned shocks (Christiano, Eichenbaum, & Evans, 2005).

On the face of it, it might appear that systematic trading strategies and DSGE models have little overlap, as systematic strategies generally rely on explicit forecasts of asset returns in the hope of generating a signal and opening a position, whilst DSGE models explain the

broader economy and are useful for drawing inferences. These models operate at a relatively low frequency in line with the rarer evolution and publishing of economic data which stems from only being measured at such frequency (often quarterly). Nevertheless, the fact that DSGE models offer the ability to forecast macroeconomic variables means that there exist datapoints to which realised values can be compared with the potential of triggering a signal. What remains on the DSGE modelling side is determining the magnitude of shocks, for which estimation techniques can be used, as well as calibrating parameters that resemble a particular economy. Appropriate assets must then be linked to the modelled macroeconomic variables for any signal to hold robustly.

Each of these assumptions and interactions can be explored further. **The intention of this paper is to lay the foundation of a new type of low-frequency systematic trading strategy based on DSGE modelling.** It offers an alternative to established long-term strategies, such as buy-and-hold.

It also does not have major barriers to entry. Coupling low dependence on data and being programmed in Python in a specific environment, it ensures reproducibility, short implementation time and ease of modification.

This framework offers further insights as a byproduct: it assesses DSGE model effectiveness by comparing IRF forecasts with realised macroeconomic values in the signal-generation phase and checks the implicit correlation between lagged asset prices and macroeconomic variables via performance of the strategies over time.

2. LITERATURE REVIEW

The development of systematic trading strategies has closely mirrored advances in technology, data availability and quantitative methods. Early systematic approaches largely automated pre-existing discretionary investment rules, allowing traders and institutions to remove emotional biases and improve execution consistency (Chan, 2013; Narang, 2013). These strategies often relied on relatively simple signals derived from technical indicators, statistical arbitrage or trend-following methodologies. Improvements in computing infrastructure and electronic market access gradually enabled higher-frequency implementation, particularly from the late 1990s onwards. This led to the growth of algorithmic and high-frequency trading strategies seeking to exploit temporary inefficiencies in market microstructure, order flow and liquidity provision (Aldridge, 2013; Hasbrouck, 2007; O'Hara, 1995).

The increasing sophistication of systematic trading has subsequently been characterised by a movement away from explicitly theory-driven models and towards empirical optimisation. Modern quantitative investment frameworks frequently rely on large datasets, machine learning techniques and statistical pattern recognition to forecast returns or classify market states (Gu, Kelly, & Xiu, 2020; López de Prado, 2018). Such approaches often prioritise predictive performance over economic interpretability, reflecting a broader trend in finance toward data-centric modelling. This has been facilitated by the expansion of alternative datasets and advances in computational power. However, despite their forecasting capabilities, many machine learning-based strategies remain vulnerable to overfitting, regime dependence and a lack of structural interpretability, particularly during periods of macroeconomic instability.

In parallel, macroeconomic modelling has evolved along a largely separate trajectory. Modern DSGE models emerged following weaknesses of aggregate metrics in macroeconomics. These models sought to explain aggregate economic fluctuations through optimising behaviour of representative agents under rational expectations and stochastic shocks. RBC models demonstrated that relatively simple economies subject to productivity shocks could reproduce several stylised features of business cycles (Kydland & Prescott, 1982; King, Plosser, & Rebelo, 1988). However, the inability of early RBC models to adequately explain nominal rigidities and monetary transmission mechanisms motivated the development of New Keynesian DSGE frameworks.

Subsequent DSGE models incorporated price stickiness, wage rigidities, monopolistic competition and monetary policy rules, substantially improving empirical performance and policy applicability (Woodford, 2003; Galí, 2015). Estimated DSGE models, particularly those of Smets & Wouters, 2003 and Smets & Wouters, 2007, became central tools within central banks and academic macroeconomics due to their ability to jointly model macroeconomic variables while maintaining structural consistency. Such models offered an internally coherent framework through which macroeconomic shocks could be identified, quantified and propagated throughout the economy. Their widespread use in monetary policy analysis reinforced DSGE modelling as a dominant paradigm in modern macroeconomics.

Despite their prominence in economics, DSGE models have rarely been integrated directly into systematic trading strategies. The majority of macro-driven systematic investment approaches instead rely on reduced-form econometric models, factor investing frameworks or purely statistical forecasting techniques. Traditional macro trading strategies often exploit relationships between interest rates, inflation, growth and asset prices using discretionary interpretation or econometric forecasting rather than structurally derived equilibrium models. Similarly, quantitative asset pricing literature has historically focused on factor models such as the Capital Asset Pricing Model (CAPM) (Sharpe, 1964) and Fama-French extensions (Fama & French, 1993; Fama & French, 2015), which explain expected returns through empirically observed risk premia rather than fully specified macroeconomic equilibria.

There exists, however, a significant body of literature linking macroeconomic conditions to financial market performance. Research has demonstrated that equity returns, bond yields and foreign exchange dynamics respond systematically to inflation surprises, monetary policy shocks and broader business cycle fluctuations (Chen, Roll, & Ross, 1986; Cochrane, 2005). Furthermore, DSGE models have increasingly been used for forecasting macroeconomic variables and analysing policy transmission, particularly through IRFs (Christiano, Eichenbaum, & Evans, 2005). This suggests the possibility that DSGE-implied forecasts or deviations between forecast and realised macroeconomic outcomes may contain exploitable information relevant to asset pricing.

Nevertheless, relatively little literature explores the use of DSGE-generated forecasts as direct trading signals within systematic trading frameworks. Where DSGE models are applied in finance, they are more commonly used for valuation, stress testing, scenario analysis or academic asset pricing applications rather than executable systematic strategies. Existing barriers likely stem from several factors. Firstly, DSGE models operate at comparatively low

frequency due to the periodicity of macroeconomic data. Secondly, calibration and estimation procedures are computationally intensive and sensitive to modelling assumptions. Thirdly, the translation of macroeconomic variables into tradable asset signals is non-trivial and may vary substantially across economies and market regimes.

This gap in the literature motivates the present study. Whilst systematic trading strategies have increasingly prioritised predictive accuracy without economic structure, DSGE models provide economically interpretable forecasts grounded in equilibrium relationships between agents in the economy. Combining these domains offers the possibility of constructing low-frequency systematic strategies that retain macroeconomic interpretability while producing investment signals upon which to trade. Furthermore, such an approach enables simultaneous evaluation of DSGE model forecasting performance and the strength of relationships between macroeconomic dynamics and financial asset returns.

The contribution of this paper therefore lies not only in proposing a novel systematic trading framework based on DSGE modelling, but also in bridging two largely disconnected strands of literature: structurally grounded macroeconomic modelling and systematic quantitative trading.

3. DATA

The DSGE trading framework in this paper is centred around the US economy and its assets due to availability of data and the prevalence of this country as a benchmark in research. The components of the framework fall into three categories in terms of data: the DSGE model, realised macroeconomic data and assets used in systematic trading strategies.

The DSGE model used in this paper is a basic RBC model. Whilst the RBC model is largely instantiated with preprepared parameters calibrated to the US economy, it requires a variable component over time which contributes towards signal generation. This is achieved by estimating an autoregressive process of order 1 (AR1) using TFP data. *Nonfarm Business Sector: Labor Productivity (Output per Hour) for All Workers* in the US (OPHNFB) from the Federal Reserve Bank of St Louis (FRED) is a quarterly seasonally adjusted metric used as TFP.

IRF forecasts of output are one of the results of running the RBC model. These are compared to the realised equivalent variable, which is quarterly seasonally adjusted *Real Gross Domestic Product* in the US (GDPC1) from FRED.

Finally, the assets sourced are the daily *S&P 500* index (^GSPC) and the daily *Dow Jones Industrial Average* index (^DJI) from Yahoo Finance, two indices composed of US equities and that are largely reflective of the US economy. In both cases, close prices are used.

FRED and Yahoo Finance data are pulled using the Fred class from the fredapi library and the yfinance library in Python, respectively. The limiting factor in terms of time horizons of the data is GDPC1 which is limited to dates after 1989-12-29 in the API, which still ensures a plethora of observations to test multiple strategy variations. All data is pulled once ahead of running any model and is instantiated using an independent data class that spans the maximum possible training and test periods. This prevents spillover between data and its handling, plus avoids having to make multiple slow connections to retrieve data that could be subject to throttling.

There is minimal data cleaning required due to the low frequency and the scrutiny applied by reputable institutions ahead of publishing. Null values are dropped and series indices are formatted to dates. US holidays as per the USFederalHolidayCalendar class in the pandas library are sourced to offset dates to nearest trading days during signal generation.

The steps performed in this section can be observed in Appendix B: DSGE Strategy Code.

3.1. Reproducibility

The design of this framework has kept ease of implementation and reproducibility at its core. The entirety of the framework has been built using Python, including data sourcing for which dedicated APIs are used.

Running the main framework script in a virtual environment with the required libraries installed is sufficient for reproducing the results in this paper. Full instructions are outlined in Appendix A: Framework Setup and in the DSGE strategy repository (Foster, 2026).

4. METHODOLOGY

The steps performed in this section can be observed in Appendix B: DSGE Strategy Code.

Signals in this strategy are fundamentally dependent on the divergence between RBC model forecasts of output and realised output. Producing forecasts requires running a particular RBC model to begin with.

4.1. RBC Model

A basic RBC model is used: it depicts the interaction between households and firms at a foundational level and is available for execution in the dolo Python library. It comes from EconForge (Winant, rbc_model.ipynb, 2019) and is defined in YAML code as per Appendix C: Basic RBC Model YAML Code. Dolo is a tool to assist researchers in solving several types of DSGE models using either local or global approximation methods (Winant, Welcome to dolo's documentation!, 2026).

In this model, households maximise expected lifetime utility as per Equation 1: Household objective:

Equation 1: Household objective

$$\max_{c_t, n_t} E_0 \sum_{t=0}^{\infty} \beta^t U(c_t, n_t)$$

where β is the discount factor, U is utility, c is consumption, n is labour and t is time.

Firms maximise profit Π as per Equation 2: Firm objective:

Equation 2: Firm objective

$$\max_{k_t, n_t} \Pi_t = y_t - w_t n_t - r_t^k k_{t-1}$$

where y is output, w is wage, r is rental rate on capital, k is capital.

These are subject to the constraints outlined in Equation 3: Budget constraint, Equation 4: Production function, Equation 5: Capital accumulation constraint and Equation 6: Exogenous TFP shock process.

Equation 3: Budget constraint

$$c_t + i_t = y_t$$

Equation 4: Production function

$$y_t = e^{z_t} k_t^\alpha n_t^{1-\alpha}$$

Equation 5: Capital accumulation constraint

$$k_{t+1} = (1 - \delta)k_t + i_t$$

Equation 6: Exogenous TFP shock process

$$z_{t+1} = \rho z_t + e_{z,t+1}$$

where i is investment, z is TFP, α is elasticity of capital, δ is depreciation of capital, ρ is persistence of the TFP shock.

Resulting equilibrium conditions and pre-calibrated parameters are coded in Appendix C: Basic RBC Model YAML Code.

For the framework to retain trading applicability, time series data entering the model remains sequential and is used for next-period forecasting. Iterating this approach along a moving window continually fine-tunes model results and applies the latest shock magnitude. This also enables strategy backtesting.

Various training lengths are employed, whereby OPHNFB productivity data is sliced in preparation for the model. This data is converted to natural logarithm form for percentage deviations and undergoes HP filtering assuming a lambda value of 1600 for quarterly data to extract the cyclical component of the time series.

The modified time series z_t is regressed against its lagged values as per the AR1 process in Equation 6: Exogenous TFP shock process using the OLS estimator. The resulting coefficient is set as the calibrated TFP persistence parameter ρ in the model. For the model to be solved quickly and given the basic RBC model type with small shocks, a local perturbation solution with a first-order Taylor approximation of the policy functions is used. An IRF is then run, assuming the latest TFP shock value $e_{z,t}$ from the residuals in the AR1 estimation.

The goal is to obtain a next-period GDP change forecast. IRF forecast values are multiplicative deviations from the normalised steady state level of 1 (as per Figure 1: Output IRF assuming $\rho=0.771$ and $e=0.0054$) and are therefore converted to percentage deviations. The output IRF array is assumed to correspond to GDP. The response at the second horizon period is recorded as the output forecast for this run of the model: the initial response

corresponds to the impact period, the response at the first horizon period reflects only the immediate labour and consumption reallocation, whilst the response at the second horizon period acknowledges capital adjustment and reflects the intended impact direction of a productivity shock. It should reflect the timing of quarterly national accounts.

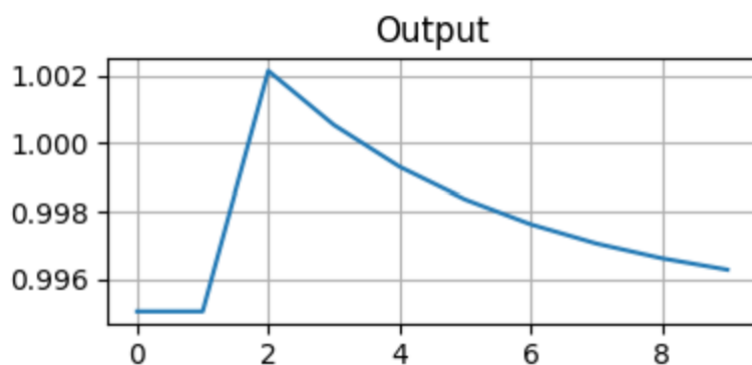


Figure 1: Output IRF assuming $\rho=0.771$ and $e=0.0054$

The process is repeated for data windows shifted quarterly within an overall horizon of the strategy backtest, as illustrated in Figure 2: Signal generation illustration. The result is a time series of next-period output forecasts.

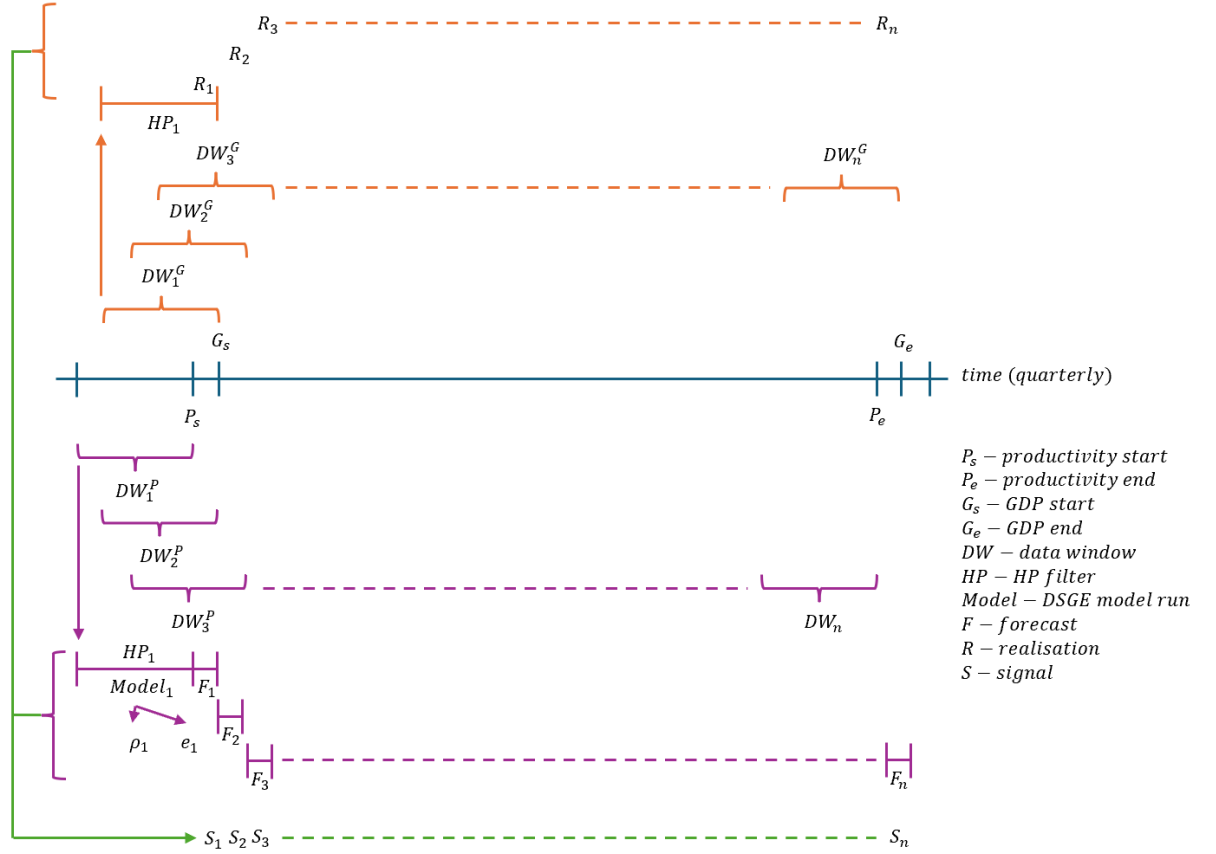


Figure 2: Signal generation illustration

4.2. Realised GDP

Realised values of GDP are needed in addition to output forecasts for subsequent signal generation. GDPC1 data undergoes HP filtering on natural logarithms to ensure a like-for-like comparison with output forecasts. It is therefore initially sliced similarly to OPHNFB data, aligning data lengths but setting the end date one quarter later than OPHNFB corresponding to the date for which the next-period forecast is made. The last element of the array is selected as the realised output value accordingly.

The process is repeated for data windows shifted quarterly, forming a time series of realised output values. This is illustrated in Figure 2: Signal generation illustration.

4.3. Signal Generation

Upon joining output forecasts with realised output, signals can be generated. Two types of signals are implemented: momentum and mean-reversion.

The momentum signal takes a long position when output forecast is greater than realised output, a short position when output forecast is less than realised output and no position

otherwise. The intuition in this scenario is that the DSGE model anticipates higher output growth than had been observed, thus a reference asset could be expected to increase in value as output catches up to close the gap.

Conversely, the mean-reversion signal takes the inverse of the momentum position with a view that new information driving divergence in realised output from the forecast implies a pending correction in equity markets and an expectation for them to price in this new information.

Ahead of signal implementation, asset data is prepared for trading. $\wedge\text{GSPC}$ and $\wedge\text{DJI}$ are used in this framework. The two indices are composed of US equities and are largely reflective of the US economy and are therefore natural candidates for the strategy. The daily series of each are restricted to the aforementioned strategy horizon, whereby the last observation ends one quarter after the last signal becomes available. This is due to a period of time required after trading a given signal for the strategy to generate profit and loss (PnL), plus the quarterly frequency of signal changes is in line with the macroeconomic data supplying the model. The series is then converted to percentage returns.

The final modification of the signal series is the crucial delay in its enforcement. In practice, macroeconomic data is released with a lag. Therefore, various lag times are tested in this strategy for the implementation to be feasible and the results to be realistic. Signals are further lagged to the nearest trading day for trade execution to be possible.

At this stage signals are joined onto the processed asset data and filled forwards. This effectively sets the position once a quarter and keeps it until the next position is established as shown in Figure 3: Positioning assuming lag=10 over one quarter. The frequency makes it a low-frequency systematic trading strategy with clear room for the assessment of whether the economic phenomena behind the momentum and mean-reversion signals hold.

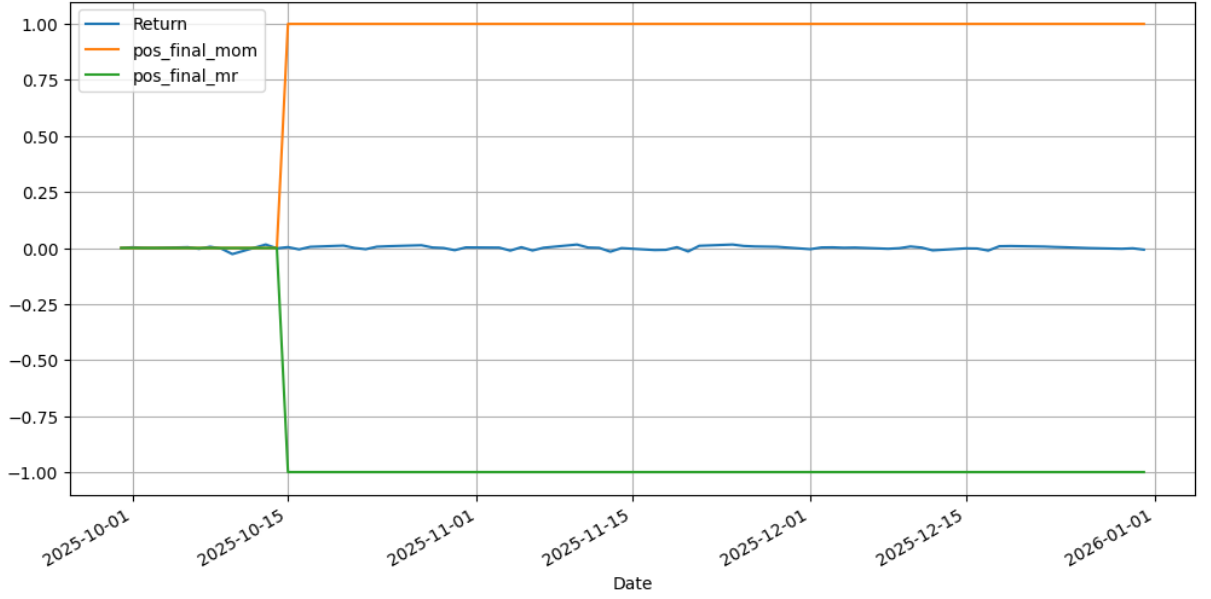


Figure 3: Positioning assuming $\text{lag}=10$ over one quarter

4.4. Extensions and Variations

The strategy is extended beyond the aforementioned setup to potentially improve performance.

Firstly, there exists a doubling down feature. This sets a signal scalar of 2 if two consecutive signals are the same at the quarterly signal generation phase, rewarding signal persistence in each of the momentum and mean-reversion approaches. For example, if output forecasts exceed realised output for two quarters, this doubles the momentum position.

Secondly, leverage can be employed. This is a multiplicative scalar passed down directly to the raw signal.

Coupling these extensions with variations mentioned throughout this section, there are numerous combinations of parameters in this strategy. These are summarised in Table 1.

Table 1: Strategy hyperparameters

Hyperparameter	Values
Model type	Basic RBC
Data length (years for model training)	5, 10, 20
Horizon	12/1999 – 9/2025 12/1999 – 12/2007 ¹

¹ 20-year data lengths are not available for these horizons, as the earliest observation available is as of 12/1989

	12/2007 – 12/2009 12/2009 – 12/2019 12/2019 – 12/2022 12/2022 – 9/2025
Signal lag (days after quarter-end)	10, 20, 30
Leverage	1, 2
Double down	False, True
Asset	S&P 500, Dow Jones Industrial Average
Strategy type	Momentum, mean-reversion

4.5. PnL Calculation and Performance Metrics

PnL is calculated daily after incorporating the asset return of the current day and the position of the previous day.

Its sum, mean and standard deviation are calculated across the strategy horizon. In addition, Sharpe ratio (SR) is calculated for a risk-adjusted view of the returns of the strategy. Daily SR is converted to annual SR, assuming 252 trading days per year as in Equation 7: Annual Sharpe ratio.

Equation 7: Annual Sharpe ratio

$$SR_{annual} = SR_{daily} \times \sqrt{252}$$

4.6. Buy-and-Hold Benchmark

As the DSGE model-inspired strategy is of low frequency and pertains to equity markets, a buy-and-hold strategy was selected as the benchmark strategy. This is also a well-established benchmark in the literature (Malkiel, 2019).

The benchmark was set up in alignment with the DSGE model strategy in terms of assets, data, and performance metrics.

5. RESULTS

TO BE UPDATED

6. LIMITATIONS AND IMPROVEMENTS

TO BE UPDATED

7. CONCLUSION

TO BE UPDATED

BIBLIOGRAPHY

- Aldridge, I. (2013). *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*. Wiley.
- Chan, E. P. (2013). *Algorithmic Trading: Winning Strategies and Their Rationale*. Wiley.
- Chen, N.-F., Roll, R., & Ross, S. A. (1986). Economic Forces and the Stock Market. *The Journal of Business*, 383–403.
- Christiano, L. J., Eichenbaum, M., & Evans, C. L. (2005). Nominal Rigidities and the Dynamic Effects of a Shock to Monetary Policy. *Journal of Political Economy*, 1–45.
- Cochrane, J. H. (2005). *Asset Pricing*. Princeton University Press.
- Fama, E. F., & French, K. R. (1993). Common Risk Factors in the Returns on Stocks and Bonds. *Journal of Financial Economics*, 3–56.
- Fama, E. F., & French, K. R. (2015). A Five-Factor Asset Pricing Model. *Journal of Financial Economics*, 1–22.
- Foster, A. (2026, May 24). *GitHub*. Retrieved from dsge_trading: https://github.com/afoster28/dsge_trading
- Galí, J. (2015). *Monetary Policy, Inflation, and the Business Cycle*. Princeton University Press.
- Gu, S., Kelly, B. T., & Xiu, D. (2020). Empirical Asset Pricing via Machine Learning. *Review of Financial Studies*, 2223–2273.
- Hasbrouck, J. (2007). *Empirical Market Microstructure*. Oxford University Press.
- King, R. G., Plosser, C. I., & Rebelo, S. T. (1988). Production, Growth and Business Cycles. *Journal of Monetary Economics*, 195–232.
- Kydland, F. E., & Prescott, E. C. (1982). Time to Build and Aggregate Fluctuations. *Econometrica*, 1345–1370.
- López de Prado, M. (2018). *Advances in Financial Machine Learning*. Wiley.
- Malkiel, B. G. (2019). *A Random Walk Down Wall Street*. W. W. Norton & Company.
- Narang, R. K. (2013). *Inside the Black Box: A Simple Guide to Quantitative and High Frequency Trading*. Wiley.
- O'Hara, M. (1995). *Market Microstructure Theory*. Blackwell Publishers.
- Sharpe, W. F. (1964). Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk. *The Journal of Finance*, 425–442.
- Smets, F., & Wouters, R. (2003). An Estimated Dynamic Stochastic General Equilibrium Model of the Euro Area. *Journal of the European Economic Association*, 1123–1175.
- Smets, F., & Wouters, R. (2007). Shocks and Frictions in US Business Cycles: A Bayesian DSGE Approach. *American Economic Review*, 586–606.

- Winant, P. (2019, June 23). *rbc_model.ipynb*. Retrieved from EconForge:
https://github.com/EconForge/dolo.py/blob/master/examples/notebooks/rbc_model.ipynb
- Winant, P. (2026, May 17). *Welcome to dolo's documentation!* Retrieved from EconForge:
<https://www.econforge.org/dolo/index.html>
- Woodford, M. (2003). *Interest and Prices: Foundations of a Theory of Monetary Policy*. Princeton University Press.

APPENDICES

Appendix A: Framework Setup

Please make sure you have all usual dependencies installed on your system. The most important ones include python, pip and ipykernel. If you're using *Visual Studio Code* for development, you should be prompted to install them automatically. Next, follow this process:

1. Create the virtual environment: `python -m venv venv`
2. Enter venv: `source venv/bin/activate` on Linux or `venv\Scripts\activate.bat` on Windows
3. Upgrade your pip: `pip install --upgrade pip`
4. Install dependencies from the requirements file: `pip install -r requirements.txt`
5. Navigate to the Model directory and open the Jupyter Notebook IDE of your choice.
Select the Python version from inside the venv you just created when prompted by ipykernel package. Suggested command: `python -m notebook`

Appendix B: DSGE Strategy Code

```
## Setup and Dependencies
```

```
import numpy as np
```

```
import pandas as pd
```

```
from pandas.tseries.holiday import USFederalHolidayCalendar
```

```
from matplotlib import pyplot as plt
```

```
from fredapi import Fred
```

```
from dolo import *
```

```
import os
```

```
from statsmodels.regression.linear_model import OLS
```

```
from pandas_datareader import data as pdr
```

```
from statsmodels.tsa.filters.hp_filter import hpfilter
```

```
import yfinance as yf
```

```
import datetime as dt
```


Model Setup

yaml_rbc = ""

name: Real Business Cycle

symbols:

exogenous: [e_z]

states: [z, k]

controls: [n, i]

parameters: [beta, sigma, eta, chi, delta, alpha, rho, zbar, sig_z]

definitions: |

$$y[t] = \exp(z[t]) * k[t]^{\alpha} * n[t]^{(1-\alpha)}$$

$$c[t] = y[t] - i[t]$$

$$rk[t] = \alpha * y[t] / k[t]$$

$$w[t] = (1-\alpha) * y[t] / n[t]$$

equations:

arbitrage: |

$$\chi * n[t]^{\eta} * c[t]^{\sigma} - w[t] \quad \perp \quad 0.0 \leq n[t] \leq \text{inf}$$

$$1 - \beta * (c[t] / c[t+1])^{\sigma} * (1 - \delta + rk[t+1]) \quad \perp \quad -\text{inf} \leq i[t] \leq \text{inf}$$

transition: |

$$z[t] = \rho * z[t-1] + e_z$$

$$k[t] = (1-\delta) * k[t-1] + i[t-1]$$

calibration:

parameters

beta : 0.99

phi: 1

delta : 0.025

alpha : 0.33

rho : 0.8

sigma: 5

eta: 1

sig_z: 0.016

zbar: 0

chi : $w/c^{\sigma}/n^{\eta}$

c_i: 1.5

c_y: 0.5

e_z: 0.0

n: 0.33

z: zbar

rk: $1/\beta - 1 + \delta$

w: $(1-\alpha) * \exp(z) * (k/n)^{\alpha}$

k: $n / (rk/\alpha)^{1/(1-\alpha)}$

y: $\exp(z) * k^{\alpha} * n^{(1-\alpha)}$

i: $\delta * k$

c: $y - i$

V: $\log(c)/(1-\beta)$

u: $c^{(1-\sigma)}/(1-\sigma) - \chi * n^{(1+\eta)}/(1+\eta)$

m: $\beta / c^{\sigma} * (1-\delta + r_k)$

kss: 10

exogenous: !UNormal

sigma: 0.01

domain:

z: $[-2 * \text{sig_z} / (1-\rho^2)^{0.5}, 2 * \text{sig_z} / (1-\rho^2)^{0.5}]$

k: $[kss * 0.5, kss * 1.5]$

options:

grid: !Cartesian

n: [100, 100]

discrete_choices: [n]

"""

file_path = "yamlDSGE/rbc.yaml"

os.makedirs(os.path.dirname(file_path), exist_ok=True)

```
with open(file_path, "w", encoding="utf-8") as f:
```

```
    f.write(yaml_rbc)
```

```
print(f'RBC model saved to {file_path}')
```

```
## Model Execution
```

```
class Modifications():
```

```
    def __init__(self):
```

```
        self.months_in_quarter = 3
```

```
    def convert_years_to_quarters(self, length_years):
```

```
        return length_years * 4
```

```
    def convert_years_to_months(self, length_years):
```

```
        return length_years * 12
```

```
    def get_shifted_date(self, date_input, date_offset_months, direction="backwards"):
```

```
        if direction == "backwards":
```

```
            date_offset_months *= -1
```

```
        shifted_date = (
```

```
            ((pd.tseries.offsets.BMonthBegin().rollback(date_input))
```

```
            +
```

```
pd.tseries.offsets.DateOffset(months=date_offset_months))
```

```
        + pd.tseries.offsets.BMonthEnd()
```

```
        ).date()
```

```

    return shifted_date

def add_buffer_for_start_offset(self, offset_months):

    return offset_months + 3 # Productivity and GDP starts need a buffer of 1 quarter, as the
    quarterly index is as of the start of the quarter rather than the end of the quarter

class Data(Modifications):

    def __init__(self, data_length_years_max, horizon_start_in, horizon_end_in):

        Modifications.__init__(self)

        self.data_length_years_max = data_length_years_max

        self.data_length_quarters_max =
self.convert_years_to_quarters(self.data_length_years_max)

        self.data_length_months_max =
self.convert_years_to_months(self.data_length_years_max)

        self.horizon_start_in = horizon_start_in

        self.horizon_end_in = horizon_end_in

        self.horizon_start = self.convert_str_date_to_dt(self.horizon_start_in)

        self.horizon_end = self.convert_str_date_to_dt(self.horizon_end_in)

        self.date_start_gdp = self.get_shifted_date(self.horizon_start,
self.add_buffer_for_start_offset(self.data_length_months_max)) # Data starts 10 years prior to
window start

        self.date_start_prod = self.get_shifted_date(self.date_start_gdp, self.months_in_quarter) #
Data starts one quarter prior to GDP data for next-period forecasting

        self.date_end_prod = self.get_shifted_date(self.horizon_end, self.months_in_quarter) #
Data ends one quarter before window end as the final forecast must be of the window end date

```

```

        # self.horizon_start_prod = self.get_shifted_date(self.horizon_start,
self.months_in_quarter) # Window starts one quarter prior to GDP data for next-period
forecasting

```

```

        self.date_end_asset = self.get_shifted_date(self.horizon_end, self.months_in_quarter,
"forwards") + pd.tseries.offsets.DateOffset(days=1) # end date in yahoo finance query is
exclusive so extend by 1 day

```

```

self.prod = self.pull_data_productivity()

```

```

self.gdp = self.pull_data_gdp()

```

```

self.asset_sp500 = self.pull_asset("^GSPC")

```

```

self.asset_dow = self.pull_asset("^DJI")

```

```

self.holidays = self.get_holidays()

```

```

def convert_str_date_to_dt(self, date_in):

```

```

    return dt.date(int(date_in[:4]), int(date_in[5:7]), int(date_in[8:]))

```

```

def convert_fred_index_to_qe(self, index_dates):

```

```

    return [date_i.date() for date_i in index_dates.date + pd.tseries.offsets.BQuarterEnd()]

```

```

# Non-farm business labor productivity index

```

```

def pull_data_productivity(self):

```

```

    prod = pdr.DataReader("OPHNFB", "fred", start=self.date_start_prod,
end=self.date_end_prod)

```

```

    prod_index_name = prod.index.name

```

```

    prod.index = self.convert_fred_index_to_qe(prod.index)

```

```

    prod.index.name = prod_index_name

```

```

prod = prod.dropna()

return prod

# Real GDP
def pull_data_gdp(self):
    gdp = pdr.DataReader("GDPC1", "fred", start=self.date_start_gdp, end=self.horizon_end)

    gdp_index_name = gdp.index.name

    gdp.index = self.convert_fred_index_to_qe(gdp.index)

    gdp.index.name = gdp_index_name

    gdp = gdp.dropna()

    return gdp

# Asset
def pull_asset(self, ticker):
    asset_ser = yf.Ticker(ticker).history(start=self.horizon_start,
end=self.date_end_asset)["Close"]

    asset_ser_name = asset_ser.index.name

    asset_ser.index = asset_ser.index.date

    asset_ser.index.name = asset_ser_name

    return asset_ser

```

```

def get_holidays(self):

    cal_us = USFederalHolidayCalendar()

    holidays = cal_us.holidays(start=self.horizon_start, end=self.date_end_asset) # holiday
    date range for the window considered for the country linked to the asset


    return holidays

class Model(Modifications):

    def __init__(self, model_type, prod, gdp, assets, asset_names, holidays):

        Modifications.__init__(self)

        self.model_type = model_type

        self.model = self.load_model(self.model_type)

        self.prod = prod

        self.gdp = gdp

        self.assets = assets

        self.asset_names = asset_names

        self.holidays = holidays


        self.latest_quarters_dict = {}

        self.output_forecasts_dict = {}

        self.latest_quarters_gdp_dict = {}

        self.outputs_realised_dict = {}


    def load_model(self, model_type):

        model = yaml_import(f"yamlDSGE/{model_type}.yaml")

```



```

return model

def generate_signals(self, data_length_years, window_gdp_start, window_gdp_end,
leverage, double_down):

    assert len(self.prod) == len(self.gdp)

    data_length_quarters = self.convert_years_to_quarters(data_length_years)

    data_length_months = self.convert_years_to_months(data_length_years)

    # Check the window and preceding data are in bounds of overall productivity and GDP
data

    earliest_datapoint_gdp = self.get_shifted_date(window_gdp_start, data_length_months)

    assert earliest_datapoint_gdp >= self.gdp.index[0], f"Outside bounds:
{earliest_datapoint_gdp} !>= {self.gdp.index[0]}"

    earliest_datapoint_prod = self.get_shifted_date(earliest_datapoint_gdp,
self.months_in_quarter)

    assert earliest_datapoint_prod >= self.prod.index[0], f"Outside bounds:
{earliest_datapoint_prod} !>= {self.prod.index[0]}"

    assert window_gdp_end <= self.gdp.index[-1], f"Outside bounds: {window_gdp_end}
!<= {self.gdp.index[-1]}"

    latest_datapoint_prod = self.get_shifted_date(window_gdp_end, self.months_in_quarter)

    assert latest_datapoint_prod <= self.prod.index[-1], f"Outside bounds:
{latest_datapoint_prod} !<= {self.prod.index[-1]}"

```

```

# Generate output forecasts

prod_index_name = self.prod.index.name

window_prod_start = self.get_shifted_date(window_gdp_start, self.months_in_quarter)

window_prod_end = self.get_shifted_date(window_gdp_end, self.months_in_quarter)

window_prod = self.prod[(self.prod.index >= window_prod_start) & (self.prod.index <=
window_prod_end)]

print("\nwindow_prod:\n")

print(window_prod)


latest_quarters = []

output_forecasts = []

for q in window_prod.index:

    q_key = str(q) + "_" + str(data_length_years) + "y"

    if q_key in self.latest_quarters_dict.keys():

        latest_quarters.append(self.latest_quarters_dict[q_key])

        output_forecasts.append(self.output_forecasts_dict[q_key])

        print(f'Reading IRF forecast as of {q} with {data_length_years} years of
calibration")

    else:

        prod_data = self.prod[(self.prod.index >= self.get_shifted_date(q,
data_length_months)) & (self.prod.index <= q)]

        assert len(prod_data.index) == data_length_quarters + 1

        prod_cycle, prod_trend = hpfilter(np.log(prod_data), lamb=1600)

```

```

# prod_cycle.plot()

a_hat = prod_cycle

a = a_hat

a_lag = a.shift()

y = a.iloc[1:]

X = a_lag.iloc[1:].values

ar1 = OLS(y.values, X).fit()

rho_hat = ar1.params[0]

print(f'Estimated rho: {rho_hat:.3f}')

eps_hat = pd.Series(

    ar1.resid,

    index=y.index,

    name="tfp_shock"

)

# eps_hat.plot()

latest_quarter = eps_hat.index[-1]

latest_shock = eps_hat.iloc[-1]

```

```

print(f'Estimated TFP shock for {latest_quarter}: {latest_shock:.4f}')

# Set the estimated TFP persistence coefficient as a calibrated parameter in the model
self.model.set_calibration(rho=rho_hat)

# print(model.calibration.flat)

#dr = time_iteration(model)

dr = perturb(self.model, order=1)

irf = response(self.model, dr, 'e_z', T=10, impulse=float(latest_shock))

# plt.figure(figsize=(8,4))

# plt.subplot(221)

# plt.plot(irf.sel(V='z'))

# plt.title('Productivity')

# plt.grid()

# plt.subplot(222)

# plt.plot(irf.sel(V='y'))

# plt.title('Output')

# plt.grid()

# plt.subplot(223)

# plt.plot(irf.sel(V='i'))

# plt.title('Investment')

# plt.grid()

```

```

# plt.subplot(224)

# plt.plot(irf.sel(V='c'))

# plt.title('Consumption')

# plt.grid()

# plt.tight_layout()


irf_output = np.array(irf.sel(V='y'))


irf_output = [(x - 1) for x in irf_output]

output_forecast = irf_output[2]

print(f'Next period IRF forecast as of {latest_quarter}: {output_forecast:.5f}')


self.latest_quarters_dict[q_key] = latest_quarter

latest_quarters.append(latest_quarter)

self.output_forecasts_dict[q_key] = output_forecast

output_forecasts.append(output_forecast)


output_forecasts_ser = pd.Series(output_forecasts, index=latest_quarters) # Next period
forecasts, e.g. 2025Q2 is forecast of 2025Q3

output_forecasts_ser.name = "output_forecast"

output_forecasts_ser.index.name = prod_index_name

output_forecasts_df = output_forecasts_ser.to_frame()

# output_forecasts_df


# Obtain realised output

```

```

gdp_index_name = self.gdp.index.name

window_gdp = self.gdp[(self.gdp.index >= window_gdp_start) & (self.gdp.index <=
window_gdp_end)]

print("\nwindow_gdp:\n")

print(window_gdp)


latest_quarters_gdp = []

outputs_realised = []

for q in window_gdp.index:

    q_key = str(q) + "_" + str(data_length_years) + "y"

    if q_key in self.latest_quarters_gdp_dict.keys():

        latest_quarters_gdp.append(self.latest_quarters_gdp_dict[q_key])

        outputs_realised.append(self.outputs_realised_dict[q_key])

        print(f"Reading realised value as of {q} with {data_length_years} years of
calibration")

    else:

        gdp_data = self.gdp[(self.gdp.index >= self.get_shifted_date(q,
data_length_months)) & (self.gdp.index <= q)]

        assert len(gdp_data.index) == data_length_quarters + 1

        gdp_cycle, gdp_trend = hpfilter(np.log(gdp_data), lamb=1600)

        output_gap = gdp_cycle

        latest_quarter_gdp = output_gap.last_valid_index()

        output_realised = output_gap.iloc[-1]

        print(f"Current period realised value as of {latest_quarter_gdp}:
{output_realised:.5f}")

```

```

self.latest_quarters_gdp_dict[q_key] = latest_quarter_gdp

latest_quarters_gdp.append(latest_quarter_gdp)

self.outputs_realised_dict[q_key] = output_realised

outputs_realised.append(output_realised)


outputs_realised_ser = pd.Series(outputs_realised, index=latest_quarters_gdp) # Realised
GDP growth after HP filter, e.g. 2025Q3 reflects GDP growth in 2025Q3

outputs_realised_ser.name = "output_realised"

outputs_realised_ser.index.name = gdp_index_name

outputs_realised_df = outputs_realised_ser.to_frame()

# outputs_realised_df


# Prepare signals

signals_init = outputs_realised_df.join(output_forecasts_df, how="outer")

output_forecasts_mod_df = signals_init.shift()[["output_forecast"]].dropna()

signals = outputs_realised_df.join(output_forecasts_mod_df)


def determine_pos(row, direction, leverage):

    if row["output_forecast"] > row["output_realised"]:

        pos = 1

    elif row["output_forecast"] < row["output_realised"]:

        pos = -1

    else:

        pos = 0

```

```

if direction == "mr":

    pos *= -1

return pos * leverage

signals["pos_mom"] = signals.apply(determine_pos, axis=1, direction="mom",
leverage=leverage)

signals["pos_mr"] = signals.apply(determine_pos, axis=1, direction="mr",
leverage=leverage)

if double_down == True:

    signals_lagged = signals[["pos_mom", "pos_mr"]].shift().fillna(0)

    signals_lagged = signals_lagged.rename(columns={"pos_mom": "pos_mom_prev",
"pos_mr": "pos_mr_prev"})

    signals = signals.join(signals_lagged)

    signals["double_down_scalar_mom"] = signals.apply(lambda row: 2 if
row["pos_mom"] == row["pos_mom_prev"] else 1, axis=1)

    signals["double_down_scalar_mr"] = signals.apply(lambda row: 2 if row["pos_mr"] ==
row["pos_mr_prev"] else 1, axis=1)

else:

    signals["double_down_scalar_mom"] = 1

    signals["double_down_scalar_mr"] = 1

signals["pos_final_mom"] = signals["pos_mom"] * signals["double_down_scalar_mom"]

```



```
signals["pos_final_mr"] = signals["pos_mr"] * signals["double_down_scalar_mr"]
```

```
return signals
```

```
def summarise_results(self, pos, pnl, strategy_type, asset_name):
```

```
    fig_pos, ax_pos = plt.subplots(figsize=(12, 6))
```

```
    pos.plot(ax=ax_pos, grid=True)
```

```
    plt.close(fig_pos)
```

```
    fig_pnl_cumsum, ax_pnl_cumsum = plt.subplots(figsize=(12, 6))
```

```
    pnl.cumsum().plot(ax=ax_pnl_cumsum, grid=True)
```

```
    plt.close(fig_pnl_cumsum)
```

```
    pnl_sum = pnl.sum()
```

```
    pnl_mean = pnl.mean()
```

```
    pnl_std = pnl.std()
```

```
    sr_daily = pnl_mean / pnl_std
```

```
    sr_annual = sr_daily * np.sqrt(252)
```

```
    results = {
```

```
        "fig_pos_" + strategy_type + "_" + asset_name: fig_pos,
```

```
        "fig_pnl_cumsum_" + strategy_type + "_" + asset_name: fig_pnl_cumsum,
```

```
        "pnl_sum_" + strategy_type + "_" + asset_name: pnl_sum,
```

```
        "pnl_mean_" + strategy_type + "_" + asset_name: pnl_mean,
```

```

        "pnl_std_" + strategy_type + "_" + asset_name: pnl_std,

        "sr_daily_" + strategy_type + "_" + asset_name: sr_daily,

        "sr_annual_" + strategy_type + "_" + asset_name: sr_annual

    }

    return results

def run_strategy(self, signals, signal_lag, window_gdp_start, window_gdp_end):

    # Delay signals by a particular number of business days to account for lag in GDP and
    productivity data availability

    signals_actual = signals[["output_realised", "output_forecast", "pos_final_mom",
                              "pos_final_mr"]]

    signals_actual.index += pd.offsets.CustomBusinessDay(n=signal_lag,
                                                           holidays=self.holidays)

    date_end_asset = self.get_shifted_date(window_gdp_end, self.months_in_quarter,
                                           "forwards")

    results = {"signals_actual": signals_actual}

    for i, asset in enumerate(self.assets):

        asset_ser = asset[(asset.index >= window_gdp_start) & (asset.index <=
date_end_asset)]

        asset_ret_ser = asset_ser.pct_change()

        asset_ret_ser.name = "Return"

        asset_ret_df = asset_ret_ser.to_frame()

```

```

asset_ret_df = asset_ret_df.join(signals_actual[["pos_final_mom",
"pos_final_mr"]]).ffill().fillna(0)

```

```

asset_ret_lagged_df = asset_ret_df[["pos_final_mom", "pos_final_mr"]].shift()

```

```

pnl = asset_ret_df[["Return"]].join(asset_ret_lagged_df).fillna(0)

```

```

pnl["pnl_mom"] = pnl["Return"] * pnl["pos_final_mom"]

```

```

pnl["pnl_mr"] = pnl["Return"] * pnl["pos_final_mr"]

```

```

results_mom = self.summarise_results(asset_ret_df["pos_final_mom"],
pnl["pnl_mom"], "mom", self.asset_names[i])

```

```

results_mr = self.summarise_results(asset_ret_df["pos_final_mr"], pnl["pnl_mr"],
"mr", self.asset_names[i])

```

```

results = results | results_mom | results_mr

```

```

return results

```

```

class BuyAndHold(Modifications):

```

```

    def __init__(self, assets, asset_names):

```

```

        Modifications.__init__(self)

```

```

        self.assets = assets

```

```

        self.asset_names = asset_names

```

```

    def run_strategy(self, window_start, window_end):

```

```

        date_end_asset = self.get_shifted_date(window_end, self.months_in_quarter, "forwards")

```

```

        results = {"signals_actual": 1}

```

```

        for i, asset in enumerate(self.assets):

```

```
asset_ser = asset[(asset.index >= window_start) & (asset.index <= date_end_asset)]
```

```
asset_ret_ser = asset_ser.pct_change().fillna(0)
```

```
asset_ret_ser.name = "Return"
```

```
pnl = asset_ret_ser
```

```
fig_pnl_cumsum, ax_pnl_cumsum = plt.subplots(figsize=(12, 6))
```

```
pnl.cumsum().plot(ax=ax_pnl_cumsum, grid=True)
```

```
plt.close(fig_pnl_cumsum)
```

```
pnl_sum = pnl.sum()
```

```
pnl_mean = pnl.mean()
```

```
pnl_std = pnl.std()
```

```
sr_daily = pnl_mean / pnl_std
```

```
sr_annual = sr_daily * np.sqrt(252)
```

```
results_bh = {
```

```
    "fig_pnl_cumsum_" + self.asset_names[i]: fig_pnl_cumsum,
```

```
    "pnl_sum_" + self.asset_names[i]: pnl_sum,
```

```
    "pnl_mean_" + self.asset_names[i]: pnl_mean,
```

```
    "pnl_std_" + self.asset_names[i]: pnl_std,
```

```
    "sr_daily_" + self.asset_names[i]: sr_daily,
```

```
    "sr_annual_" + self.asset_names[i]: sr_annual
```

```
}
```

```

results = results | results_bh

return results

data_10y_19991231_20250930 = Data(10, "1999-12-31", "2025-09-30")

# rbc = Model(

#   "rbc",

#   data_20y_19991231_20250930.prod,

#   data_20y_19991231_20250930.gdp,

#   data_20y_19991231_20250930.asset,

#   data_20y_19991231_20250930.holidays

# )

model_types = ["rbc"]

data_lengths = [5, 10, 20]

windows = [

  [dt.date(1999, 12, 31), dt.date(2025, 9, 30)],

  [dt.date(1999, 12, 31), dt.date(2007, 12, 31)],

  [dt.date(2007, 12, 31), dt.date(2009, 12, 31)],

  [dt.date(2009, 12, 31), dt.date(2019, 12, 31)],

  [dt.date(2019, 12, 31), dt.date(2022, 12, 31)],

  [dt.date(2022, 12, 31), dt.date(2025, 9, 30)],

]

signal_lags = [10, 20, 30]

leverage_cases = [1, 2]

```

```

double_down_cases = [False, True]

results = {}

for model_type in model_types:

    rbc = Model(

        model_type,

        data_10y_19991231_20250930.prod,

        data_10y_19991231_20250930.gdp,

        [data_10y_19991231_20250930.asset_sp500,
data_10y_19991231_20250930.asset_dow],

        ["SP", "DOW"],

        data_10y_19991231_20250930.holidays

    )

    bh = BuyAndHold(

        [data_10y_19991231_20250930.asset_sp500,
data_10y_19991231_20250930.asset_dow],

        ["SP", "DOW"]

    )

    for window in windows:

        window_str = str(window[0]) + "_" + str(window[1])

        results[window_str] = {}

        for data_length in data_lengths:

```

```

for signal_lag in signal_lags:

    for leverage in leverage_cases:

        for double_down in double_down_cases:

            if not ((data_length == 20) & (window[0] < dt.date(2009, 12, 31))): # GDP data
is unavailable before 1989-12-29 so avoid sourcing data before then

                signals = rbc.generate_signals(data_length, window[0], window[1], leverage,
double_down)

                strategy = rbc.run_strategy(signals, signal_lag, window[0], window[1])


                key = model_type + "_" + str(data_length) + "cal_" + window_str + "_" +
str(signal_lag) + "lag"

                if leverage > 1:

                    key += "_" + str(leverage) + "lev"

                if double_down == True:

                    key += "_dd"

                results[window_str][key] = {"signals": signals, "strategy": strategy}


strategy_bh = bh.run_strategy(window[0], window[1])


key_bh = "bh_" + window_str

results[window_str][key_bh] = {"strategy": strategy_bh}

results

main_results_sorted = {}

```

```
for window in windows:
```

```
    window_str = str(window[0]) + "_" + str(window[1])
```

```
    main_results_sorted[window_str] = {}
```

```
    pnls_sp = []
```

```
    srs_sp = []
```

```
    pnls_dow = []
```

```
    srs_dow = []
```

```
    for strategy in results[window_str].keys():
```

```
        if "bh" in strategy:
```

```
            pnls_sp.append([strategy, results[window_str][strategy]["strategy"]["pnl_sum_SP"]])
```

```
            srs_sp.append([strategy, results[window_str][strategy]["strategy"]["sr_annual_SP"]])
```

```
            pnls_dow.append([strategy,  
results[window_str][strategy]["strategy"]["pnl_sum_SP"]])
```

```
            srs_dow.append([strategy, results[window_str][strategy]["strategy"]["sr_annual_SP"]])
```

```
        else:
```

```
            pnls_sp.append([strategy + "_mom",  
results[window_str][strategy]["strategy"]["pnl_sum_mom_SP"]])
```

```
            pnls_sp.append([strategy + "_mr",  
results[window_str][strategy]["strategy"]["pnl_sum_mr_SP"]])
```

```
            srs_sp.append([strategy + "_mom",  
results[window_str][strategy]["strategy"]["sr_annual_mom_SP"]])
```

```
            srs_sp.append([strategy + "_mr",  
results[window_str][strategy]["strategy"]["sr_annual_mr_SP"]])
```



```

        pnls_dow.append([strategy + "_mom",
results[window_str][strategy]["strategy"]["pnl_sum_mom_DOW"]])

        pnls_dow.append([strategy + "_mr",
results[window_str][strategy]["strategy"]["pnl_sum_mr_DOW"]])

        srs_dow.append([strategy + "_mom",
results[window_str][strategy]["strategy"]["sr_annual_mom_DOW"]])

        srs_dow.append([strategy + "_mr",
results[window_str][strategy]["strategy"]["sr_annual_mr_DOW"]])


    main_results_sorted[window_str]["pnls_sp_sorted"] = sorted(pnls_sp, key=lambda x: x[1],
reverse=True)

    main_results_sorted[window_str]["srs_sp_sorted"] = sorted(srs_sp, key=lambda x: x[1],
reverse=True)


    main_results_sorted[window_str]["pnls_dow_sorted"] = sorted(pnls_dow, key=lambda x:
x[1], reverse=True)

    main_results_sorted[window_str]["srs_dow_sorted"] = sorted(srs_dow, key=lambda x: x[1],
reverse=True)

main_results_sorted.keys()

main_results_sorted["1999-12-31_2025-09-30"]["pnls_sp_sorted"]
main_results_sorted["1999-12-31_2007-12-31"]["pnls_sp_sorted"]
main_results_sorted["2007-12-31_2009-12-31"]["pnls_sp_sorted"]
main_results_sorted["2009-12-31_2019-12-31"]["pnls_sp_sorted"]
main_results_sorted["2019-12-31_2022-12-31"]["pnls_sp_sorted"]
main_results_sorted["2022-12-31_2025-09-30"]["pnls_sp_sorted"]

```

```
results["1999-12-31_2025-09-30"]["rbc_5cal_1999-12-31_2025-09-30_10lag"]["strategy"]["fig_pnl_cumsum_mom_SP"]
```

Appendix C: Basic RBC Model YAML Code

name: Real Business Cycle

symbols:

exogenous: [e_z]

states: [z, k]

controls: [n, i]

parameters: [beta, sigma, eta, chi, delta, alpha, rho, zbar, sig_z]

definitions: |

$$y[t] = \exp(z[t]) * k[t]^{\alpha} * n[t]^{(1-\alpha)}$$

$$c[t] = y[t] - i[t]$$

$$rk[t] = \alpha * y[t] / k[t]$$

$$w[t] = (1-\alpha) * y[t] / n[t]$$

equations:

arbitrage: |

$$\chi * n[t]^{\eta} * c[t]^{\sigma} - w[t] \quad \perp \quad 0.0 \leq n[t] \leq \text{inf}$$

$$1 - \beta * (c[t] / c[t+1])^{\sigma} * (1 - \delta + rk[t+1]) \quad \perp \quad -\text{inf} \leq i[t] \leq \text{inf}$$

transition: |

$$z[t] = \rho * z[t-1] + e_z$$

$$k[t] = (1-\delta) * k[t-1] + i[t-1]$$

calibration:

parameters

beta : 0.99

phi: 1

delta : 0.025

alpha : 0.33

rho : 0.8

sigma: 5

eta: 1

sig_z: 0.016

zbar: 0

chi : $w/c^{\sigma}/n^{\eta}$

c_i: 1.5

c_y: 0.5

e_z: 0.0

n: 0.33

z: zbar

rk: $1/\beta - 1 + \delta$

w: $(1-\alpha) * \exp(z) * (k/n)^{\alpha}$

k: $n/(rk/\alpha)^{1/(1-\alpha)}$

y: $\exp(z)*k^\alpha*n^{1-\alpha}$

i: $\delta*k$

c: $y - i$

V: $\log(c)/(1-\beta)$

u: $c^{1-\sigma}/(1-\sigma) - \chi*n^{1+\eta}/(1+\eta)$

m: $\beta/c^\sigma*(1-\delta+rk)$

kss: 10

exogenous: !UNormal

sigma: 0.01

domain:

z: $[-2*\sigma_z/(1-\rho^2)^{0.5}, 2*\sigma_z/(1-\rho^2)^{0.5}]$

k: $[kss*0.5, kss*1.5]$

options:

grid: !Cartesian

n: [100, 100]

discrete_choices: [n]